SMKC CIVIC INNOVATION FORUM 2026

A JOINT INITIATIVE OF SMKC AND WALCHAND COLLEGE OF ENGINEERING, SANGLI

IMPLEMENTATION TECHNOLOGY STACK

ROUND 2 (FINAL STAGE)

Technical framework for shortlisted teams

Technology Stack

COMPONENT	PROPOSED STANDARD
Frontend — Web	Next.js preferred; HTML5/CSS3/JavaScript or an equivalent modern framework permitted
Mobile App	React Native / Flutter
Backend	Node.js / .NET
API	REST API
Development Database	MySQL / PostgreSQL / SQL Server / Oracle — team's choice
Database Architecture	Database-independent; keep database-specific logic to a minimum
Data Access	Backend Repository / Data Access Layer / ORM
AI / Computer Vision	Python, OpenCV, YOLO, TensorFlow / PyTorch or a suitable framework
GIS / Maps	OpenStreetMap
Authentication Controller	Backend-controlled authentication + Role-Based Access Control (RBAC) + JWT + CORS
Integration	REST APIs for future SMKC ERP / other-system integration
Source Control	Git / GitHub / GitLab or equivalent
Final Hosting	Existing SMKC server infrastructure
Deployment Approval	Final deployment subject to SMKC IT Department compatibility and security testing

1. Core Architecture

The following architecture is mandatory for all teams:

LAYER	FLOW
Application	Web / Mobile Application
Interface	REST API / JSON
Processing	Backend / Business Logic Layer
Data Access	Data Access / Repository Layer
Storage	SQL Database

Architecture rule: The frontend/mobile application must never access the database directly.

2. Database Policy

During development, teams may use PostgreSQL, MySQL, SQL Server or Oracle according to their convenience. However, the solution must be designed so it can subsequently be migrated to Oracle Database 12c.

DATABASE SHOULD PRIMARILY HANDLE	BACKEND SHOULD PRIMARILY HANDLE
• Data storage • Tables and relationships • Primary / foreign keys • Constraints • Indexes • Transactions • Standard CRUD operations	• Business logic • Validation • Workflow and escalation • Calculations • Approval logic • AI processing • Application rules • Integration logic

Migration-friendly practice: Minimize stored procedures, database functions, triggers, proprietary SQL, PostgreSQL/MySQL-specific features and database-specific business logic. Document any unavoidable use.

3. AI

For Pothole, GVP and Illegal Hoarding detection, teams may select an appropriate AI stack such as Python + OpenCV + YOLO / TensorFlow / PyTorch.

AI development and training do not need to happen on the Windows Server 2012 server. Teams may develop and train models in their own development environments.

INPUT / IMAGE	AI MODEL	DETECTION RESULT	BACKEND API	DATABASE
Image or input	Inference	Prediction / detection	API processing	Store result

Final handover must include the trained model, model weights, source code, dependencies and inference procedure.

4. GIS / Location

Since almost all five challenges have a location component, use a common set of basic fields:

LATITUDE	LONGITUDE	TIMESTAMP	ACCURACY
Latitude value	Longitude value	Date and time	Location accuracy

Preferred mapping approach: OpenStreetMap + Leaflet wherever feasible. Avoid making the core solution dependent on paid Google Maps or another paid third-party service unless specifically approved. Basic location storage should not depend on PostgreSQL/PostGIS; store normal latitude/longitude values to keep migration to Oracle manageable.

5. Track-wise Technology

CHALLENGE	MAJOR TECHNOLOGY
Smart Pothole Detection & Escalation	AI / Computer Vision + GPS + REST API + Web Dashboard
Smart GVP Detection & Littering Enforcement	AI / Computer Vision + Geo-tagging + Workflow + Dashboard
Geo-tagged Photo Attendance & Field Presence	Mobile App + Camera + GPS + Backend API + RBAC
Garbage Vehicle GPS Tracking & Citizen View	GPS + REST API + Maps + Live / Periodic Tracking + Citizen Web View
Illegal Hoarding Detection & Digital Encroachment	AI / Image Processing + GPS + Inspection / Notice Workflow + Dashboard

6. Mandatory Security Requirements

Every team should follow at least the following security requirements:

- HTTPS for API communication.
- Role-Based Access Control (RBAC) — Admin, Officer, Field User, Citizen, etc., as applicable.
- Passwords must be securely hashed.
- No database passwords or API keys inside source code.
- Input validation and parameterized database queries.
- File/image upload validation.
- Audit trail for important transactions.
- Login/session security.
- Proper API authentication.
- Production Oracle database must not be directly exposed to the Internet.
- Personal/official SMKC data must not be uploaded to unauthorized public AI/cloud services.
- Separate configuration for Development / Test / Production.

7. Mandatory Final Deliverables

Every shortlisted team should submit the complete project, not only a demonstration.

NO.	DELIVERABLE
01	Complete source code
02	Frontend source code
03	Backend / API source code
04	Mobile app source code, if applicable
05	AI source code and trained model/weights, if applicable
06	Complete database schema and SQL scripts
07	Sample/test database
08	ER diagram
09	API documentation — preferably Swagger/OpenAPI
10	Software/library dependency list with versions
11	Installation and deployment guide
12	User manual
13	Administrator manual
14	Source-code build instructions
15	Third-party API/library details and licences
16	Details of any paid services/API/subscription
17	Database-specific features used, if any
18	Test credentials and sample data
19	Final working demonstration